

AS-REP roasting y Kerberoasting

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Son dos ataques que piden tickets de kerberos y se quedan con la parte cifrada para crackearla offline. AS-REP roasting funciona contra usuarios con la preautenticación desactivada y no necesita credenciales. Kerberoasting pide tickets de cuentas de servicio (con SPN) y requiere una credencial de dominio cualquiera.

El ticket va cifrado con una clave derivada de la password del usuario o de la cuenta de servicio. Si esa password es débil, la crackeas offline sin tocar el DC (sin riesgo de bloqueo). Las cuentas de servicio suelen tener passwords viejas y privilegios altos: premio doble.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

AS-REP: en cuanto tengas una lista de usuarios (smb-enum), aunque no tengas password. Kerberoast: en cuanto consigas UNA credencial válida. Ambos encajan en la fase AD (ad-modelo).

Cómo identificarla

- GetNPUsers devuelve hashes \$krb5asrep\$ (hay usuarios sin preauth).
- GetUserSPNs lista cuentas de servicio con SPN y devuelve \$krb5tgs\$.
- Los nombres de esas cuentas huelen a privilegio (svc_sql, backup, admin).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Lanza GetNPUsers -no-pass (AS-REP) o GetUserSPNs -request con una credencial (Kerberoast).

Evidencia esperada: Un hash \$krb5asrep\$ o \$krb5tgs\$ en la salida. Sin hash, ese vector concreto no aplica en este dominio.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
impacket-GetNPUsers $DOMAIN/ -usersfile users.txt -no-pass -dc-ip $IP
```

Objetivo y contexto: AS-REP roasting sin credenciales: pide tickets para usuarios con preauth desactivada. Solo necesitas la lista de usuarios y la IP del DC.

Resultado esperado: Hashes \$krb5asrep\$ para los usuarios vulnerables. Cada uno es crackeable offline (hashcat -m 18200). Sin salida = nadie tiene preauth desactivada.

Capa 2: prueba documentada

```
impacket-GetUserSPNs $DOMAIN/$USER:$PASS -dc-ip $IP -request
```

Objetivo y contexto: Kerberoasting: con una credencial de dominio pides los tickets de las cuentas de servicio. Esas cuentas suelen tener passwords debiles y muchos permisos.

Resultado esperado: Hashes \$krb5tgs\$ junto al nombre del servicio. Prioriza los que parezcan admin; crackea con hashcat -m 13100.

Anatomía de los payloads

La primera prueba es `impacket-GetNPUsers $DOMAIN/ -usersfile users.txt -no-pass -dc-ip $IP`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un hash \$krb5asrep\$ o \$krb5tgs\$ en la salida. Sin hash, ese vector concreto no aplica en este dominio.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- AS-REP: pasa la lista de usuarios a GetNPUsers con -no-pass.
- Kerberoast: con una credencial, pide los tickets de SPN con GetUserSPNs -request.
- Crackea los hashes con hashcat (modo 18200 AS-REP, 13100 Kerberoast) y rockyou.
- Cada password crackeada -> nueva identidad: vuelve a enumerar el dominio con ella.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- AS-REP: ningún usuario tiene la preautenticación desactivada (GetNPUsers no devuelve nada).

- Kerberoast: no hay cuentas de servicio con SPN registrado en el dominio.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Dos formas de sacar hashes crackeables de Kerberos: sin credenciales (AS-REP) o con una cualquiera (Kerberoast). La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y preconditiones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (asrep-kerberoast)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.